[TOPIC 19] 2강. FCN & U-Net

※ 최종 업데이트: 2026. 06. 12.

지난 시간에는 Segmentation 이 무엇인지에 대해 전반적으로 배웠습니다. 한 가지 의문이 있다면, 우리는 이미 이미지를 분류하는 CNN 을 만들 수 있는데, 그 CNN 을 조금만 손보면 픽셀마다 클래스를 부여할 수 있지 않을까요?

결론부터 말하면, 분류 CNN 을 그대로 쓸 수는 없어요. 구조상 픽셀 단위 예측에 안 맞는 부분이 있거든요. 이번 강의에서는 그 문제가 무엇인지 짚고, 그걸 해결하며 Segmentation 의 포문을 연 두 모델 **FCN**과 **U-Net**을 만나봅니다.

이번 강의에서는 아래 내용에 대해 함께 알아보겠습니다.

2-1. 분류 CNN 의 한계

2-2. FCN

2-3. U-Net

2-4. 두 모델 정리

2-1. 분류 CNN 의 한계

1) 분류 CNN 이 하는 일

이미지를 분류하는 CNN 이 어떻게 생겼는지 떠올려볼게요. 입력 이미지가 들어오면, 합성곱과 풀링을 거치며 **특징 맵의 크기가 점점 줄어들죠.** 224×224 였던 이미지가 112×112, 56×56 으로 작아지는 대신, "이 이미지에 어떤 특징이 있는지" 가 필터에 차곡차곡 쌓입니다. 그리고 마지막에 **Fully Connected Layer** 를 거쳐 "고양이 90%, 강아지 10%" 같은 라벨 하나를 내놓게 되죠.

이 구조의 핵심은 **공간 정보를 점점 버리고, 대신 "무엇인지" 만 남긴다**는 거예요. 마지막에 라벨 하나만 답하면 되니까, "고양이가 사진의 어디에 있었는지" 는 중요하지 않아요. 오히려 위치에 상관없이 고양이를 알아보는 게 분류에는 유리하죠.

2) Segmentation 의 요구 조건

그런데 Segmentation 은 정반대예요. **픽셀마다 답**을 해야 하니까, "이 위치에 무엇이 있는지" 라는 공간 정보가 끝까지 살아 있어야 합니다. 게다가 출력도 라벨 하나가 아니라 **입력과 같은 크기의 결과**여야 해요. 224×224 이미지를 넣으면, 224×224 짜리 "픽셀별 클래스 지도" 가 나와야 하죠.

여기서 분류 CNN 을 Segmentation 에 적용할 수 없는 두 가지 이유가 드러나요.

- **Fully Connected Layer 의 존재** — 모든 픽셀을 한 줄로 펴서 섞어버리니, 위치 정보가 사라져요.
- **크기가 줄어들기만 하는 구조** — 작아진 특징 맵을 다시 원래 크기로 되돌릴 방법이 없어요.

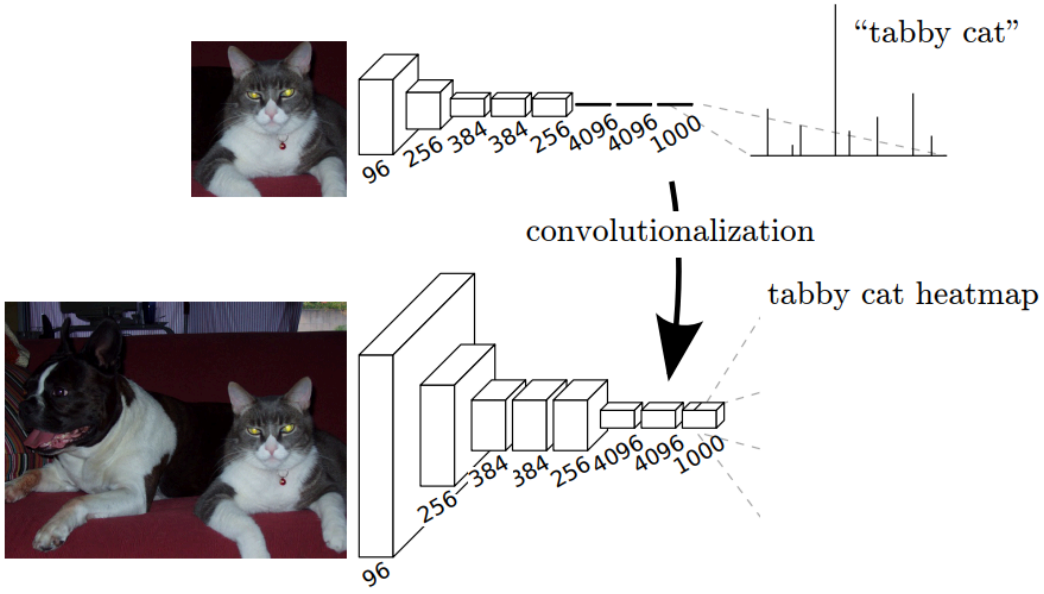
이 두 이유를 해결하면 Segmentation 모델을 만들 수 있게 됩니다. 그리고 그걸 FCN 이 해냈어요.

2-2. FCN

FCN(Fully Convolutional Network) 은 2015년에 나온 모델입니다. 이름이 곧 핵심 아이디어예요. "Fully Convolutional", 즉 온통 합성곱으로만 만들었다는 뜻입니다.

1) 합성곱화(Convolutionalization)

FCN 의 첫 번째 아이디어는 분류 CNN 의 마지막에 있던 Fully Connected Layer 를 없애고, 그 자리를 합성곱으로 채운 것입니다. 이걸 **합성곱화(Convolutionalization)** 라고 부릅니다.



<Fully Convolutional Networks for Semantic Segmentation (2015)>

Convolutionalization. 공간 정보를 손실하게 하는 FC Layer 를 모두 합성곱 층으로 대체하였다.

사실 Fully Connected Layer 와 합성곱은 **같은 계산**입니다. "전부 연결한다 vs 필터로 덮는다" 가 달라 보이지만, 가중치를 어떻게 곱해 더하는지 따라가 보면 똑같아요.

특징 맵이 `[512, 7, 7]` 까지 작아졌고, 여기서 **어떤 클래스에 대한 Logit 값 하나**를 뽑는다고 해볼게요.

- **Fully Connected** — 7×7×512 = 25088개를 한 줄로 펴고, 가중치 25088개와 하나씩 곱해 전부 더합니다. ($\hat{y} = WX + b$) `점수 = w1·x1 + w2·x2 + ... + w25088·x25088 + b`
- **합성곱** — 필터 크기를 **특징 맵과 똑같은 7×7** 로 잡아 선언합니다 (`nn.Conv2d(in_channels=512, out_channels=1, kernel_size=7)`). 계산해보면 이 레이어의 필터들은 총 7×7×512 = 25088개의 가중치를 가져요. 필터가 특징 맵 전체를 **딱 한 번** 덮고, 대응되는 25088개의 값을 하나씩 곱해 더합니다. `점수 = w1·x1 + w2·x2 + ... + w25088·x25088 + b`

두 식이 완전히 동일합니다. 가중치 개수도 25088개로 같고, 곱하고 더하는 방식도 같아요. **"전체를 덮는 큰 필터" 가 곧 Fully Connected** 인 셈이죠.

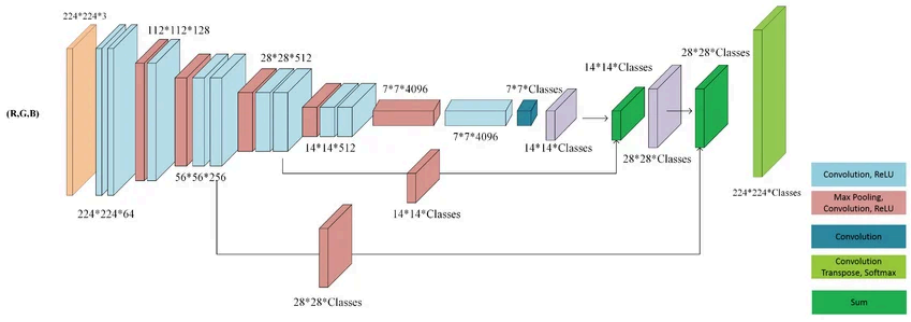
핵심은 필터 **크기를 특징 맵 크기와 같게 맞췄다**는 데 있어요. 보통 합성곱은 작은 필터(예: 3×3)가 이미지 위를 **옮겨 다니며 여러 번** 찍지만, 필터를 특징 맵 전체 크기로 키우면 옮겨 다닐 자리가 없어 **한 번** 덮고 끝납니다. 이 "한 번 덮기" 가 바로 "Fully Connected 한 번 통과" 와 같은 거예요. 차이는 계산이 아니라, 결과를 **한 줄로 펴서 내보내느냐(위치 정보 손실)** 아니면 **공간 모양을 유지한 채 내보내느냐(위치 정보 보존)** 뿐입니다.

그래서 Fully Connected 를 동등한 합성곱으로 바꾸면, 위치 정보가 끝까지 살아남습니다. 앞서 본 첫 번째 문제(위치를 뭉갠다)가 해결되는 셈이에요.

덤으로 따라오는 이점도 있어요. Fully Connected Layer 는 입력 크기가 고정이라 늘 224×224 같은 정해진 크기만 받을 수 있었는데, 합성곱은 **입력이 더 커도 그냥 작동**합니다. 큰 이미지를 넣으면 출력도 그만큼 커져서, 위치별로 클래스 점수가 담긴 **특징 맵 형태**로 나와요. 분류기가 자연스럽게 픽셀 예측기로 바뀌는 순간입니다.

2) 인코더-디코더 구조

크기가 줄어들기만 하는 구조가 분류 모델을 Segmentation 에 사용할 수 없는 이유였습니다. 그러면 크기를 늘려주면 되고 우리는 이미 그 구조에 대해서 배운 바 있죠. 바로 **인코더-디코더(Encoder-Decoder) 구조**입니다.



FCN 의 구조 (FCN-8s)

- **인코더(Encoder)** — 이미지를 점점 줄이며 특징을 뽑아내요. 분류 CNN 의 앞부분과 똑같아요. "무엇이 있는지" 를 압축하는 단계입니다.

- **디코더(Decoder)** — 줄어든 특징 맵을 **다시 원래 크기로 키웁니다.** 이렇게 키우는 과정을 **업샘플링(Upsampling)** 이라고 해요.

인코더에서 작게 압축한 정보를, 디코더에서 다시 펼쳐 픽셀별 답을 만드는 것입니다. 들어올 때 크기로 답이 나오니, 픽셀마다 클래스를 칠한 지도를 얻을 수 있습니다.

업샘플링은 오토인코더에서 이미 만난 적이 있어요. 디코더가 압축된 표현을 원래 이미지 크기로 되돌릴 때 쓴 **Transposed Convolution** 을 기억하실 겁니다. FCN 의 디코더도 똑같은 방식으로, 학습 가능한 Transposed Convolution 을 써서 작아진 특징 맵을 키웁니다. 7×7 까지 작아진 특징 맵을 단계적으로 키워 입력과 같은 크기로 되돌리는 거죠.

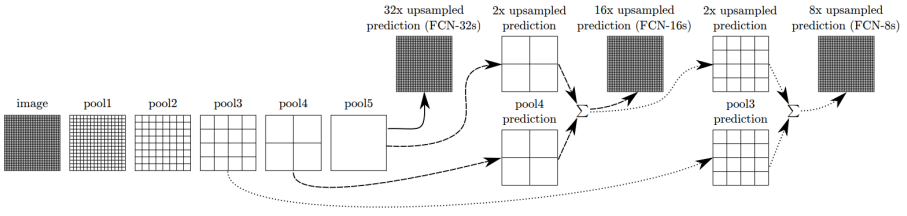
오토인코더에서는 이미지를 압축했다가 **같은 이미지로** 복원했다면, 여기서는 복원의 목적지가 달라요. 픽셀마다 클래스를 칠한 **Segmentation Map** 으로 되돌리는 거예요. 키우는 도구(Transposed Convolution)는 같지만, 무엇을 목적하느냐가 다른 셈입니다.

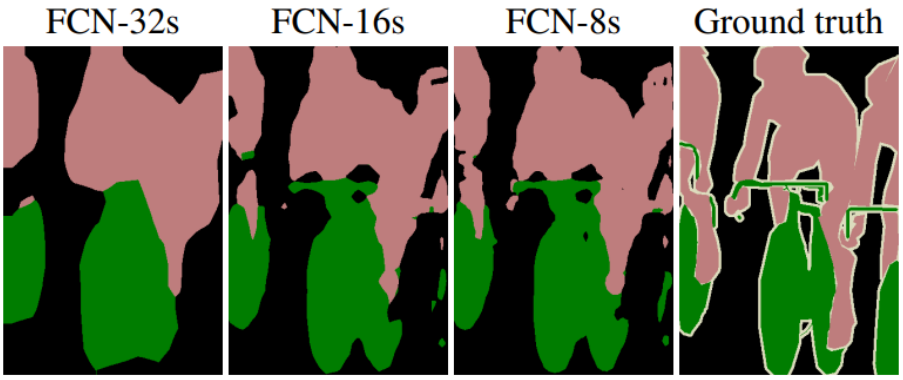
그런데 업샘플링 과정에 약간의 문제가 있었어요. 인코더를 끝까지 거친 특징 맵은 원본보다 한참 작아진 상태예요. 224×224 이미지가 7×7 까지 줄었다면 무려 32배가 작아진 거죠. 이 7×7 을 **한 번에** 224×224 로 키우면 7×7 의 칸 하나가 원본의 32×32 픽셀 영역을 통째로 대표하게 되어서, 32×32 짜리 큰 덩어리가 죄다 같은 값으로 칠해집니다. 작은 썸네일을 무리하게 확대했을 때 모자이크처럼 뭉개지는 것과 똑같아요. 그래서 이렇게 한 번에 키운 결과는 경계가 뭉툭합니다. FCN 에서는 이 버전을 **FCN-32s**라고 불러요. 32배를 한 번에 키웠다는 뜻입니다.

3) Skip Connection

업샘플링에서의 문제를 해결하기 위해, **Skip Connection** 이라는 개념을 도입합니다. ResNet 에서 본 것과 비슷한데, 그 때는 입력 값을 레이어를 거친 후의 출력 값에 더해주는 형태였다면, 이번엔 인코더의 중간 결과를 알맞게 가져와서 더해주는 형태예요. 인코더가 이미지를 줄여온 과정을 떠올려보면 224 → 112 → 56 → 28 → 14 → 7 처럼 단계를 거쳤는데, 이 중간 단계들(28×28, 14×14)은 아직 경계 디테일이 살아 있거든요. 끝까지 줄어든 7×7 보다 훨씬 또렷합니다.

그래서 FCN 은 인코더의 중간 결과를 디코더로 **건너뛰어(Skip) 전달** 해, 키우는 도중에 디테일을 더해줍니다. 이 연결을 **Skip Connection**이라고 해요. 빌려오는 단계를 늘릴수록 결과가 또렷해져요.





Skip Connection 의 단계에 따른 결과 퀄리티 변화

- **FCN-32s** — 7×7 을 한 번에 32배로 키움. 가장 거칠어요.
- **FCN-16s** — 7×7 을 2배만 키운 뒤 인코더의 14×14 단계를 더하고, 그 다음 16배로 키움. 한 단계 또렷해져요.
- **FCN-8s** — 거기서 한 번 더. 28×28 단계까지 더한 뒤 8배로 키움. 경계가 가장 또렷합니다.

왜 더하는 게 도움이 될까요? 깊은 단계(7×7)는 많이 줄어 뭉툭하지만 "무엇인지" 를 잘 알고, 얕은 단계(28×28)는 "경계가 어디인지" 를 잘 알아요. 이 둘을 합치면 의미도 알고 경계도 또렷한 결과가 나옵니다.

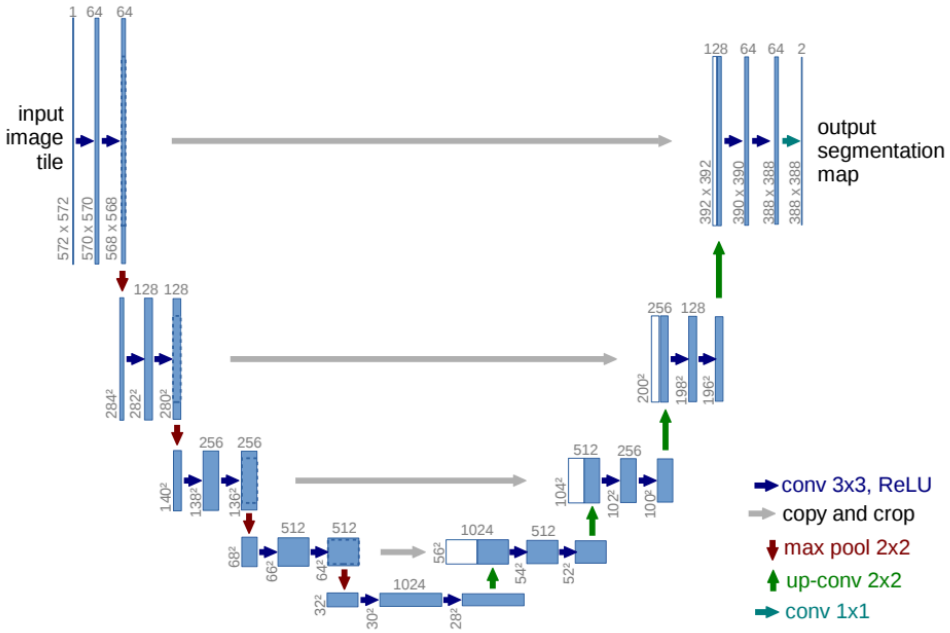
FCN 은 픽셀 단위 예측의 길을 열었지만 아쉬움도 있었어요. 업샘플링이 비교적 단순하고, Skip Connection 도 몇 단계만 써서 세밀한 경계를 살리는 데 한계가 있었죠. 이 아쉬움을 개선한 모델이 바로 U-Net 입니다.

2-3. U-Net

U-Net은 FCN 이 나온 같은 해(2015년)에 등장한, FCN 의 발전형이에요. 흥미롭게도 **의료 영상**을 위해 만들어졌어요. 세포나 장기의 경계를 정밀하게 따야 하는 의료 영상에서는, FCN 의 뭉툭한 경계로는 부족했거든요.

1) U 자 모양 구조

U-Net 이라는 이름은 모델의 생김새에서 왔어요. 인코더와 디코더가 **좌우 대칭**으로 마주 보는 구조라, 그림으로 그리면 알파벳 U 처럼 생겼거든요.



<U-Net: Convolutional Networks for Biomedical Image Segmentation (2015)>

- **왼쪽 내리막(인코더)** — 이미지를 줄이며 특징을 뽑는 길이에요. 합성곱과 풀링을 반복하며 크기를 절반씩 줄이고 채널을 늘려가요. "무엇이 있는지" 를 점점 압축하는, 분류 CNN 의 앞부분과 같은 역할입니다.
- **바닥(Bottleneck)** — U 자의 가장 아래, 제일 많이 압축된 지점이에요. 공간 크기는 가장 작지만 "이 이미지가 무엇에 관한 것인지" 라는 가장 추상적이고 압축된 의미를 담고 있어요.
- **오른쪽 오르막(디코더)** — 바닥에서 다시 위로 올라오며 이미지를 키우는 길이에요. 업샘플링으로 크기를 두 배씩 키우면서, 픽셀마다 클래스를 칠한 답을 만들어갑니다.

여기까지는 FCN 의 인코더-디코더와 큰 틀이 같아요. 차이는 Skip Connection 의 쓰임에 있어요.

2) 발전된 형태의 Skip Connection

FCN 이 Skip Connection 을 일부 단계에만 썼다면, U-Net 은 **모든 단계에서** 씁니다. 인코더의 각 단계와 디코더의 같은 높이 단계를 **빠짐없이 연결**한 것입니다.

이를 통해 디코더가 이미지를 키울 때, 같은 높이의 인코더에서 **그 크기였을 때의 디테일**을 그대로 받아올 수 있어요. 큰 그림은 디코더가 그리고, 세밀한 경계는 인코더에서 받아온 정보로 채우는 식입니다. 그래서 U-Net 은 경계를 훨씬 또렷하게 살려냅니다. U 자의 양쪽을 잇는 수많은 가로줄이 바로 이 Skip Connection 입니다.

그리고 Connection 을 연결하는 방식도 약간 다릅니다. FCN 은 인코더의 정보를 디코더에 **더해버린(Add) 한편,** U-Net 은 더하는 대신 **이어붙입니다(Concatenate)**. 전자의 경우는 두 정보를 한 덩어리로 섞어버리

게 되고, 후자는 인코더에서 온 특징과 디코더의 특징을 나란히 쌓아두는 겁니다. 섞지 않고 둘 다 보존하니, 뒤따르는 합성곱이 "어느 쪽 정보를 얼마나 쓸지" 스스로 학습해서 골라 쓸 수 있어요.

여기에 더해, U-Net 의 디코더는 단순히 키우기만 하지 않아요. 한 단계 키울 때마다 이어붙인 정보를 합성곱으로 다시 다듬습니다. 인코더와 대칭을 이루는, 똑같이 깊은 디코더인 거죠. FCN 의 디코더가 비교적 단순했던 것과 대비되는 지점이에요.

3) 의료 분야에서 강한 이유

U-Net 은 **적은 데이터에서도** 경계를 잘 잡아냅니다. 이 특징이 의료 분야의 니즈와 딱 맞아떨어졌어요. 의료 영상은 환자 데이터라 모으기 어렵고, 라벨링에도 전문가가 필요해서 데이터가 늘 부족하거든요.

적은 데이터로도 좋은 성능을 낼 수 있는 이유는 두 가지입니다. 하나는 앞서 본 **Skip Connection**, 인코더의 디테일을 디코더가 직접 받아 쓰니, 적은 데이터로도 경계를 정교하게 살릴 수 있게 됩니다. 다른 하나는 **강한 데이터 증강**입니다. U-Net 논문은 이미지를 회전하거나 늘이고 비트는 변형을 적극적으로 적용해, 한 장의 데이터를 여러 장처럼 부풀려 학습했어요. 구조와 증강이 함께 맞물려 "데이터가 적어도 잘 되는" 모델이 된 거예요.

그래서 U-Net 은 지금도 의료 영상 Segmentation 하면 가장 먼저 떠오르는 모델이고, 우리가 다음 실습에서 직접 만들어볼 모델이기도 합니다.

> ⭐ **[BONUS] U-Net 의 계보를 이은 모델들**
> U-Net 은 워낙 성능이 좋았기 때문에, 이후 수많은 변형이 나왔어요. 두 가지만 살짝 맛보면:
>
> - **U-Net++** — Skip Connection 을 더 촘촘하게 만든 버전이에요. 인코더와 디코더를 바로 잇는 대신, 그 사이를 작은 합성곱 단계들로 촘촘히 메워서 정보가 더 매끄럽게 전달되도록 했어요.
> - **Attention U-Net** — Skip 으로 정보를 넘길 때 **중요한 영역에 집중(Attention)** 하도록 한 버전이에요. 배경처럼 덜 중요한 부분은 약하게, 찾으려는 물체 영역은 강하게 넘겨서 더 정확하게 분할해요.
>
> 기본 구조는 모두 U-Net 의 인코더-디코더 + Skip Connection 을 그대로 물려받아요. U-Net 하나를 제대로 이해하면, 이런 변형들도 "어디를 어떻게 손봤는지" 로 자연스럽게 읽혀요.

오늘은 Segmentation 의 대표 모델 두 가지를 살펴봤어요. 주요 내용을 다시 한 번 짚어보도록 하죠.

- **분류 CNN 의 한계** — Fully Connected Layer 가 위치 정보를 뭉개고, 크기가 줄기만 해서 픽셀 단위 예측에 맞지 않음.
- **FCN** — Fully Connected 를 합성곱으로 바꿔(Convolutionalization) 위치 정보를 살리고, 인코더-디코더로 줄였다가 다시 키움. Skip Connection 으로 디테일을 보완하지만, 단계적으로 키워도 경계가 다소 뭉툭함.

- **U-Net** — FCN 의 발전형. 모든 단계에서 Skip Connection 을 쓰고, 더하는 대신 이어붙여 (Concatenate) 정보를 풍부하게 넘김. 대칭형 디코더로 경계를 또렷하게 살려냄.
- **U-Net 과 의료 영상** — Skip Connection 과 강한 데이터 증강 덕분에 적은 데이터로도 잘 학습돼, 데이터가 부족한 의료 분야에서 특히 강함.
- **공통 아이디어** — 줄이며 특징을 뽑고(인코더), 다시 키우며 픽셀별 답을 만들고(디코더), Skip Connection 으로 잃어버린 디테일을 되살린다.

다음 시간에는 지금까지와 결이 다른 모델을 만나봅니다. Semantic Segmentation 이 픽셀을 클래스로 칠하는 일이었다면, 개체까지 구분하는 **Instance Segmentation** 은 어떻게 풀 수 있을까요? 객체 인식에서 배운 박스를 활용하는 **Mask R-CNN** 이 그에 대한 해답입니다. 다음 시간에 뵙겠습니다.